Build a Large Language Model from Scratch

This thesis is a summary of [1] and reports on the experience with it.

REPORT 3

Coding Attention Mechanisms

Giorgi Gogsadze
Revaz Goguadze

Supervised by Prof. Walter Tichy

Structure

1. **What is an Attention Mechanism**
2. **The plan to Implement a Complete Attention Mechanism**
3. **Simplified self-attention without trainable weights**
4. **Self-attention with trainable weights**
5. **Causal attention that hides future words**
6. **Multi-head attention**

Except for the exercise solutions, all code and data examples presented in this work are adapted from Chapter 3 of [1].

What is an Attention Mechanism

Before a formal explanation of an attention mechanism, we present a useful analogy: *The attention mechanism can be compared to using a highlighter while reading. The reader does not pay equal attention to every single word on a page so they focus on the most important words to understand the main idea*.

In the same way, an attention mechanism helps a large language model focus on the most relevant parts of an input sentence to better understand its meaning and context, and perform a task.

Without attention, RNNs suffer from information bottlenecks. By trying to force an entire sequence into one hidden state, they frequently lose context in complex sentences with distant dependencies.

How does the attention mechanism solve this problem?

The attention mechanism addresses this by giving the model access to all parts of the original text at every step. When the model is generating a new token, it can look back at the entire input, pay attention to the most relevant parts and decide which words are most important for predicting the next word in the output.

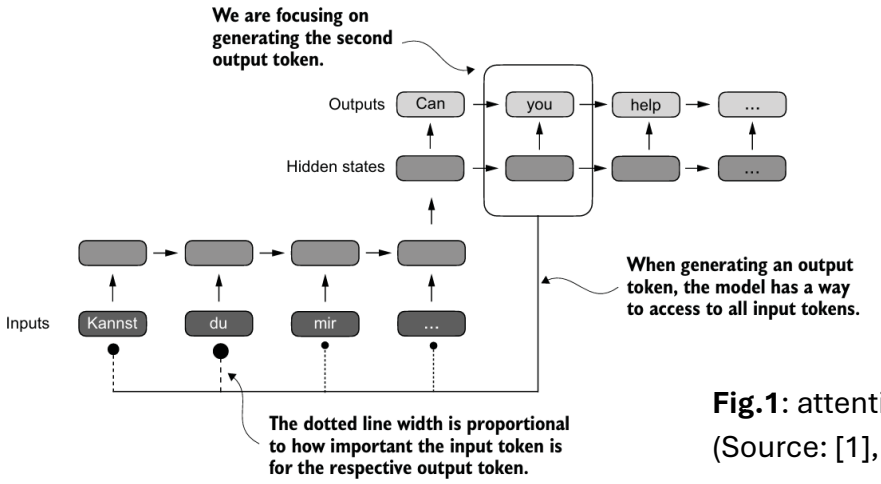

Fig.1: attention mechanism (Source: [1], p. 54)

Why is it important in implementing LLMs?

As a result of the attention mechanism, the model can weigh the importance of every piece of incoming information to determine which parts are most relevant at any given moment.

The plan to Implement a Complete Attention Mechanism

In this report we show how to achieve compact and efficient implementation of multi-head attention that we can then plug into the LLM architecture. The implementation will follow four key stages:

1. **Simplified Self-Attention:** Transforms input token embeddings into context vectors. A context vector is different from a regular embedding, as it is enriched with information from all other tokens in the sequence, which lets the model understand the meaning of a token in a specific sentence rather than just generally.
2. **Self-Attention with trainable weights:** Represents relationships between tokens (Query, Key, Value).
3. **Causal Attention:** Applies a mask to the attention scores to prevent the model from accessing future information in a sequence.
4. **Multi-Head Attention:** Organizes the attention mechanism into multiple parallel heads, which makes it possible for the model to capture different types of relationships and patterns in the data simultaneously.

Simplified self-attention without trainable weights

The way the simplified self-attention mechanism works is straightforward: compute a context vector for every input token, where the result represents the original token with additional information from all other tokens in the sequence.

The Detailed Process:

We assume that each input token is represented by a fixed-length embedding vector, encoding semantic information about the word.

1. **Compute Attention Scores ω:**

   Take one specific input token as a current query (q_i). The current query is the token for which we are computing the context vector right now.

   For a given query q_i, an attention score $\omega_{i,j}$ is computed with respect to every token x_j by taking the dot product of their embedding vectors:

   $$\omega_{i,j} = q_i \cdot x_j$$

   The dot product acts as a similarity measure between embeddings: higher values indicate stronger alignment between the two tokens. This measure is closely related to cosine similarity.

In practice, computing attention scores for all queries and all tokens results in an attention score matrix, which can be efficiently computed as a matrix multiplication:

```
attn_scores = inputs @ inputs.T
```

2. **Normalize via Softmax:**

   The raw attention scores are converted into attention weights $\alpha_{i,j}$ using the softmax function:

$$\alpha_{i,j} = \text{softmax}(\omega_{i,j})$$

   Softmax normalizes the scores so that the weights for each query sum to 1. These weights can be interpreted as the relative importance of all tokens with respect to the current query token.

3. **Calculate Context Vectors Z:**

   After computing attention weights between the query token q_i and all input tokens, each input embedding is multiplied by its corresponding weight, and the resulting vectors are summed to form the final context vector for q_i.

$$z_i = \sum_j \alpha_{i,j} x_j$$

   This operation combines information from all tokens while emphasizing those most relevant to the query. Figure 2 illustrates this process for the second input token:

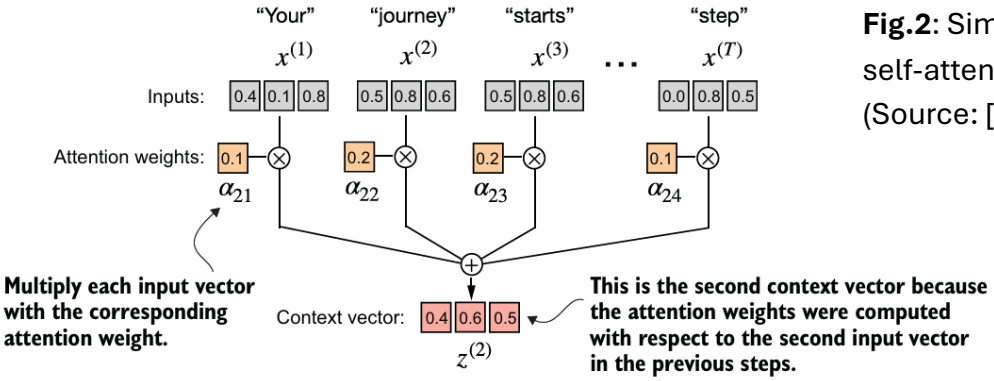

Fig.2: Simplified self-attention (Source: [1], p. 61)

This process is repeated for every token in the sequence as the query. As a result, each token attends to every other token, producing a complete set of context vectors and naturally forming a matrix of attention weights.

With this we can obtain context vectors for each input that now slightly reflects the importance of other inputs. But we can improve it with trainable weights.

Self-attention with trainable weights

While the simplified self-attention mechanism demonstrates the core concept, its major limitation is that the attention scores are fixed, preventing the model from learning.

We need to make a strategy, so that instead of comparing raw input embeddings directly, the model learns how to look at the embeddings. As introduced in [2], this is achieved by projecting each input embedding into three different learned representations:

- **Query (q_i)** — represents what the current token is *looking for*
- **Key (k_i)** — represents how each token can be *matched*
- **Value (v_i)** — represents the *information content* of each token

To get these representations, three different weight matrices (W_Q, W_K and W_V) are initialized with **random values** and optimized during training using gradient descent. Each token embedding x_i is then projected as:

$$q_i = x_i W_Q, \quad k_i = x_i W_K, \quad v_i = x_i W_V$$

Dimensionality reduction and learned projections

As explained in [2], query, key, and value projections are often performed in lower-dimensional spaces, particularly in multi-head attention, where each head operates on a reduced representation. Therefore, even here we reduce output dimension d_{out} as:

- It reduces computational cost,
- It enables multiple attention heads to operate in parallel,
- It allows the model to learn compact, task-relevant representations.

Computation steps

Figure 3 illustrates the full computation path of self-attention with trainable weights.

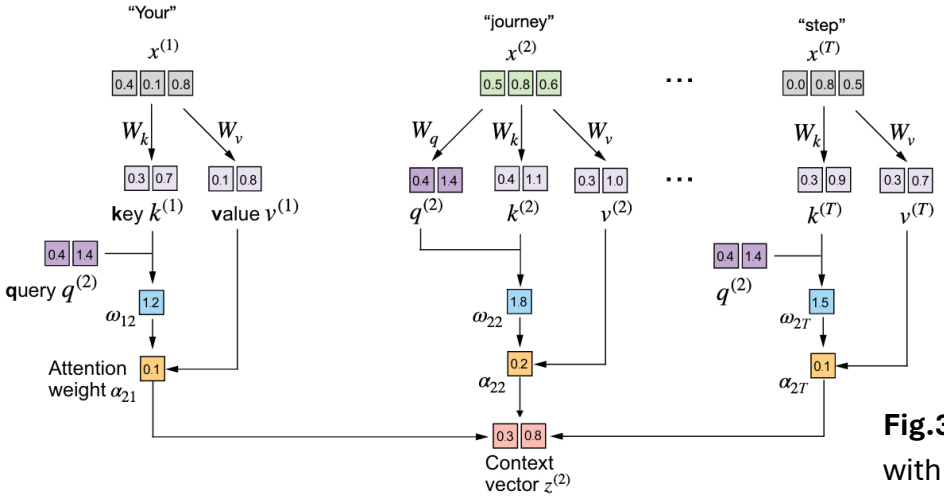

Fig.3: self-attention with trainable weights (Source: [1], p. 69)

Step 1: Project inputs into Q, K, and V

Each input embedding is multiplied by the corresponding weight matrix:

```
queries = inputs @ W_query
keys = inputs @ W_key
values = inputs @ W_value
```

Step 2: Compute scaled attention scores

For each query token, attention scores are computed using the dot product with all keys:

$$\omega_{i,j} = q_i \cdot k_j$$

The scores are scaled by $\sqrt{d_k}$ to prevent large values that would destabilize training:

```
attn_scores = queries @ keys.T
attn_scores_scaled = attn_scores / d_k**0.5
```

Step 3: Normalize to attention weights

The scaled scores are converted into attention weights using softmax. Each row of the resulting matrix sums to 1 and represents the relative importance of all tokens with respect to the current query.

Step 4: Compute context vectors

The context vector for a query token is computed as a weighted sum of value vectors:

$$z_i = \sum_j \alpha_{i,j} v_j$$

This operation integrates information from all tokens, emphasizing those most relevant to the query.

Exercise 3.1

We can see that even though nn.Linear in SelfAttention_v2 uses a different weight initialization scheme than nn.Parameter(torch.rand(d_in, d_out)) used in SelfAttention_v1, both implementations are otherwise similar. We can transfer the weight matrices from a SelfAttention_v2 object to a SelfAttention_v1, such that both objects then produce the same results:

```
sa_v1.W_query = torch.nn.Parameter(sa_v2.W_query.weight.T)
sa_v1.W_key = torch.nn.Parameter(sa_v2.W_key.weight.T)
sa_v1.W_value = torch.nn.Parameter(sa_v2.W_value.weight.T)
```

With these trainable weights, the self-attention mechanism can now learn context-dependent relationships from data, forming the basis for the more advanced attention structures used in LLMs.

Causal attention that hides future words

For language generation tasks, a model must predict the next word based only on the words that have come before it. Standard self-attention allows every token to see every other token in the sequence, so the model can simply copy the next word instead of learning to predict it.

Causal attention (also known as masked attention) ensures that when calculating the context vector for a given token, the model can only attend to the current token and all previous tokens in the sequence. This is achieved by applying a mask to the attention weights (as shown in Figure 3) or to the attention scores before the softmax function is applied.

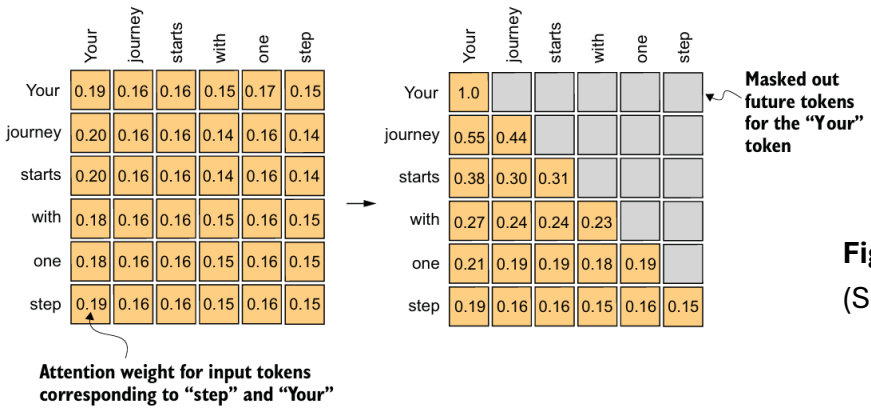

Fig.4: causal attention (Source: [1], p. 74)

The Mask:

The most efficient way to implement the causal mask is to replace the attention scores for all future positions with -inf.

Softmax Result:

When the softmax function is applied, $lim_{x \to -\infty}\ e^x = 0$, effectively nullifying the influence of any future token on the current one. The process builds directly on the previous self-attention mechanism.

Regularization/Dropout: We also apply dropout to the attention weights to reduce overfitting during training

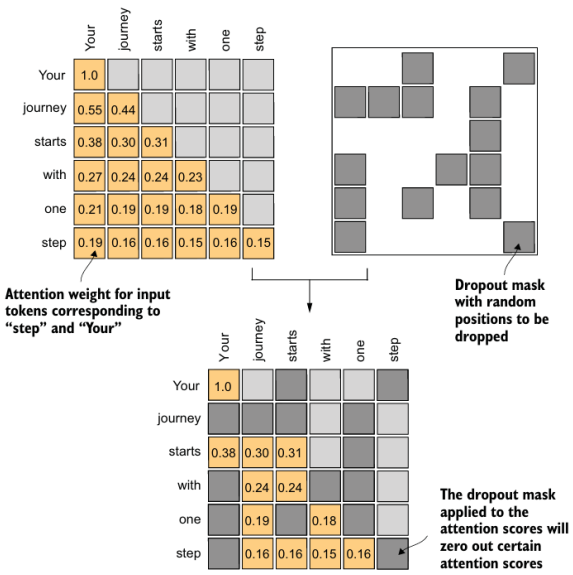

Fig.5: causal attention with dropouts
(Source: [1], p. 79)

Regularization acts as a training filter. By randomly switching off attention weights, it prevents the model from relying on specific words too heavily. This forces the network to learn multiple redundant paths for the same information, ensuring it **generalizes** well to new, unseen text.

Multi-head attention

While the causal self-attention mechanism is powerful, it is limited to learning only one type of relationship between tokens per layer. Multi-head attention enhances this by running the attention mechanism multiple times in parallel, each with its own **set of learned weight matrices**. This allows the model to simultaneously focus on different aspects of the input. For example, one head might learn syntactic relationships while another learns semantic ones.

The outputs from these parallel heads are then combined to produce a final, richer output that captures a more comprehensive understanding of the text.

Conceptual Implementation: Stacking Attention Heads

Conceptually, the simplest way to implement multi-head attention is to stack multiple CausalAttention modules. Each head operates independently with its own weights, and their individual outputs are concatenated to form the final result. This approach is intuitive but less efficient as it involves running several separate computations.

Exercise 3.2

This is how we can use the class so that output context vectors are two-dimensional:

```
# Set d_out=1 so that when concatenated
# the final dimension is d_out * num_heads = 1 * 2 = 2.
context_length = batch.shape[1]
d_in, d_out = 3, 1
mha = MultiHeadAttentionWrapper(d_in, d_out, context_length, 0.0, 2)
context_vecs = mha(batch)
```

Efficient Implementation: The Standard Approach

A more computationally efficient implementation avoids running separate loops or modules. Instead, it performs one large matrix multiplication for all heads at once and then splits the results into the respective heads using tensor reshaping. This is the standard approach used in transformer architectures.

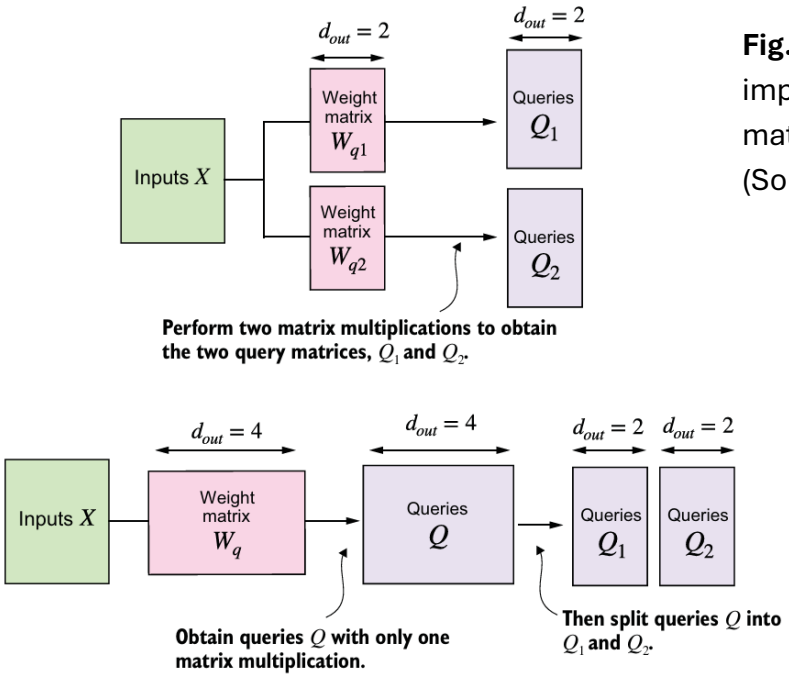

Fig.6: difference between 2 implementations: wrapping vs matrix multiplying (Source: [1], p. 88)

The process involves these key steps:

1. **Combined Projection:** Project the inputs into one large matrix containing queries, keys, and values for all heads combined. The output dimension (d_out) must be divisible by the number of heads (num_heads).
2. **Splitting into Heads:** Reshape and transpose the Q, K, and V matrices to separate them into multiple heads. A tensor of shape (batch, num_tokens, d_out) is reshaped into (batch, num_heads, num_tokens, head_dim), where head_dim = d_out / num_heads.
3. **Parallel Attention Calculation:** Perform the scaled dot-product attention calculation simultaneously across all heads.
4. **Concatenating Heads:** Reshape the output context vectors to merge the heads back together.
5. **Final Projection:** Apply a final linear layer (out_proj) to the combined output, allowing the model to learn how to best mix the information from the different heads.

Encapsulation in a Final MultiHeadAttention Class

The Multi-Head Attention module simply puts everything we discussed so far into one clean unit.

Each word first creates three versions of itself: what it is looking for (query), what it represents (key), and what information it carries (value). Using these, the word decides which earlier words matter most to it.

This process doesn't happen just once. It happens multiple times in parallel, in different "heads". Each head looks at the sentence in a slightly different way, so the model can notice different patterns at the same time.

A mask makes sure a word can't see the future, only itself and what came before. The important words are weighted more, their information is combined, and each word gets an updated, context-aware representation.

At the end, all heads are merged back together into one final vector per word.

Exercise 3.3

Using the MultiHeadAttention class, we can initialize a multi-head attention module that has 12 attention heads, input and output embedding sizes of 768 dimensions and context length of 1,024 tokens matching the configuration of the smallest GPT-2 model:

```
context_length = 1024
d_in, d_out = 768, 768
num_heads = 12
mha = MultiHeadAttention(d_in, d_out, context_length, 0.0, num_heads)
```

This efficient MultiHeadAttention module is the complete, production-ready version that will serve as a foundational building block for the full LLM architecture.

References

[1] Raschka, S. (2024). *Build a large language model (from scratch)*. Shelter Island, NY: Manning Publications.

[2] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.